

DeforestWatch-DRC ? Guide complet & mode d'emploi

Surveillance et prediction de la deforestation de la foret equatoriale du Bassin du Congo (province du Mai-Ndombe, RDC) par imagerie satellite et Machine Learning.

Projet de fin d'etudes ? L3 Data Science, Universite Protestante au Congo (FASI). Ce document explique TOUT le projet et sert de mode d'emploi pas a pas.

1. Presentation du projet

La RDC abrite la deuxieme plus grande foret tropicale humide du monde. DeforestWatch-DRC exploite les images satellites Sentinel-2 et des modeles de Machine Learning pour detecter, quantifier et predire la deforestation, et fournit une plateforme d'aide a la decision.

Ce que fait le projet

- Detecter les zones deforestees par classification d'images (5 classes de couverture).
- Quantifier la perte forestiere annee par annee (2015-2025).
- Comparer trois modeles : Random Forest, XGBoost, U-Net (CNN).
- Predire les zones a risque de deforestation future.
- Servir une API (FastAPI) et deux interfaces (dashboard Streamlit + frontend React).

Mode demonstration vs donnees reelles

Le projet fonctionne CLE EN MAIN grace a un mode demonstration : des donnees satellites synthetiques realistes permettent de tout faire tourner sans compte Google Earth Engine. Quand vous disposez de vraies images, vous basculez en un clic (voir section 6).

2. Architecture et arborescence

deforestwatch/	
config/	Configuration centralisee (settings.py)
src/	
data/	Collecte GEE, meteo, sources (demo/reel), provider
preprocessing/	Indices spectraux, masquage nuages, mosaïques, PySpark
models/	Random Forest, XGBoost, U-Net, trainer, evaluator, risque
visualization/	Cartes (Folium), graphiques (Plotly)
api/	API FastAPI (auth 2FA/JWT, routes, base de donnees)
utils/	Logger, helpers, generateur synthetique
streamlit_app/	Dashboard (login, dashboard, analyse, predictions, admin)
frontend/	Frontend React (Vite + Tailwind)
notebooks/	01 EDA, 02 pretraitement, 03 modelisation, 04 resultats
scripts/	Automatisation (collecte, entraînement, export, docs)
tests/	Suite pytest (29 tests)
data/	raw/ (vraies images) - processed/ - models/
docs/	Memoire, slides, ce guide

3. Stack technique

Composant	Technologie
Langage	Python 3.11
Collecte satellite	Google Earth Engine, rasterio
Big Data	PySpark
ML classique	scikit-learn, XGBoost
Deep Learning	TensorFlow / Keras (U-Net)

Composant	Technologie
API	FastAPI (JWT + 2FA)
Dashboards	Streamlit + React (Vite/Tailwind)
Base de donnees	PostgreSQL / Supabase (repli SQLite)
Conteneurisation	Docker, docker-compose
Tests / CI	pytest, GitHub Actions

4. Installation

```
# 1. Cloner le depot
git clone https://github.com/joycewetu0805/deforestwatch.git
cd deforestwatch

# 2. Environnement virtuel
python -m venv venv
source venv/bin/activate          # Windows : venv\Scripts\activate

# 3. Dependances
pip install -r requirements.txt

# 4. Configuration
cp .env.example .env              # editez si besoin (DEMO_MODE=true par default)
```

5. Demarrage rapide (mode demo)

Aucune cle API requise. En quatre commandes :

```
make seed      # genere les donnees de demo + entraine les modeles
make api       # API FastAPI      -> http://localhost:8000/docs
make dashboard # Dashboard Streamlit -> http://localhost:8501
make frontend  # Frontend React   -> http://localhost:5173
```

Comptes de demonstration (dashboard)

Email	Mot de passe	Code 2FA	Role
admin@deforestwatch.cd	admin123	123456	admin
user@deforestwatch.cd	user123	123456	utilisateur

6. Donnees : basculer entre demo et reel

C'est le point central pour passer du prototype aux vraies images. Trois facons, de la plus simple a la plus permanente :

Methode	Action	Portee
Toggle dashboard	Barre laterale > Source de donnees (Auto/Demo/Reelle)	A chaud, sans redemarrer
API (admin)	POST /api/v1/admin/source/{auto demo real}	Process API en cours
Persistant (.env)	make demo / make real / make mode	Tous les demarrages suivants

Mode Auto : utilise les vraies images si presentes dans data/raw/, sinon la demo. Si vous demandez 'reel' sans donnees disponibles, l'application fait un repli transparent sur la demo.

Format attendu des vraies images (data/raw/)

```
data/raw/
```

```
composites/2015.tif    # 6 bandes Sentinel-2 : B2,B3,B4,B8,B11,B12 (reflectance)
composites/2016.tif    # une image par annee
landcover/2015.tif     # (optionnel) classes 0..4 = verite terrain
topography.tif         # 3 bandes : altitude, pente, aspect
```

Classes de couverture : 0=Forêt dense, 1=Forêt dégradée, 2=Agriculture/Sol nu, 3=Eau, 4=Urbain/Bati. Toutes les images d'une zone doivent partager la même grille.

```
make check-data    # verifie annees, bandes, alignement, etiquettes
```

7. Telecharger les vraies donnees depuis Google Earth Engine

C'est la fonctionnalité que vous aviez en v1 : récupérer directement les images Sentinel-2. Le script `scripts/gee_export.py` s'en charge.

Etape 1 ? Authentification GEE

Creez un compte Earth Engine (gratuit pour usage académique) sur <https://earthengine.google.com>, puis authentifiez-vous :

```
earthengine authenticate    # ouvre le navigateur, colle le jeton
# OU compte de service (sans interaction) : renseignez dans .env
#   GEE_SERVICE_ACCOUNT=...@...iam.gserviceaccount.com
#   GEE_KEY_FILE=config/gee-key.json
```

Etape 2 ? Lancer l'export

```
# Option A : export vers Google Drive (pleine resolution 10 m, recommande)
python -m scripts.gee_export --drive
#   -> cree une tache par annee ; suivez-les sur
#       https://code.earthengine.google.com/tasks
#   -> telechargez ensuite les GeoTIFF depuis Drive vers data/raw/composites/

# Option B : telechargement direct (zone reduite / apercu rapide)
python -m scripts.gee_export --download --scale 100
#   -> ecrit directement data/raw/composites/{annee}.tif

# Choisir les annees (par default 2015..2025)
python -m scripts.gee_export --download --years 2023 2024 2025 --scale 60
```

Etape 3 ? Tester SANS compte GEE

Pour valider toute la chaîne réelle sans authentification, générez des GeoTIFF synthétiques au bon format :

```
make export-demo    # ecrit data/raw/composites + landcover + topography
make check-data     # verifie le jeu de donnees
# puis dans le dashboard : toggle 'Donnees reelles' (ou make real)
```

Etape 4 ? Basculer en mode reel

```
make real           # DEMO_MODE=false dans .env
make train          # re-entraîne les modeles sur les vraies images
make dashboard      # le dashboard lit désormais data/raw/
```

8. Les interfaces

API FastAPI (port 8000)

Documentation interactive Swagger sur <http://localhost:8000/docs>. Principaux endpoints :

- POST `/api/v1/auth/register`, `/api/v1/auth/login`, `/api/v1/auth/verify-otp` ? authentification + 2FA
- GET `/api/v1/statistics` ? perte forestière par année
- GET `/api/v1/predictions/{year}` ? synthèse des zones à risque
- GET `/api/v1/models` ? performances des modèles

- GET /api/v1/source ? source de donnees active (demo/reel)
- GET /api/v1/maps/landcover/{year} ? carte de couverture (PNG) d'une annee
- GET /api/v1/maps/risk ? carte de risque (PNG)
- GET /api/v1/alerts ? alertes de deforestation detectees (filtres severite/annee)
- GET /api/v1/alerts/summary ? synthese des alertes
- POST /api/v1/reports ? signaler une deforestation (ouvert a tous)
- GET /api/v1/admin/reports ? signalements recus (admin)
- POST /api/v1/admin/source/{mode} ? bascule demo/reel (admin)
- GET /api/v1/admin/users, /admin/logs ? back-office (admin)

Dashboard Streamlit (port 8501)

- Connexion securisee (email + mot de passe + code 2FA).
- Dashboard : KPIs, carte, evolution forestiere, top secteurs touches.
- Analyse : classification par annee, comparaison, matrice de confusion du modele.
- Predictions : carte de risque, zones critiques.
- Alertes : detection automatique de deforestation par secteur + formulaire de signalement citoyen (persiste en base).
- Admin : utilisateurs, logs, metriques, modeles (role admin uniquement).
- Barre laterale : toggle Source de donnees (demo / reel) a chaud.

Frontend React (port 5173)

- Landing page 'CongoForest Watch' : hero anime, KPIs, fonctionnalites, stack.
- Machine a remonter le temps : time-lapse animee de la deforestation 2015-2025 avec lecture automatique et comparateur avant/apres (glissiere 2015 vs annee).
- Dashboard de monitoring : KPIs, courbes, carte de risque (fetch /api/v1/statistics).
- Lancement : cd frontend && npm install && npm run dev

9. Modeles de Machine Learning

Modele	Type	Entree
Random Forest	Pixel (baseline)	13 features (6 bandes + 4 indices + 3 topo)
XGBoost	Pixel (boosting)	13 features
U-Net (CNN)	Segmentation	Tuiles 128x128x6
Risk Predictor	Risque binaire	Distances routes/villages, pente, voisinage

Entrainement : make train (rapide) ou make train avec l'option complete. Metriques : Accuracy, Precision, Recall, F1, AUC-ROC, IoU par classe, Mean IoU, matrice de confusion. Le rapport est ecrit dans data/processed/model_metrics.json.

10. Tests et integration continue

```
make test          # 29 tests pytest (pretraitement, modeles, API, sources)
pytest tests/ -v --cov=src  # avec couverture
```

GitHub Actions (.github/workflows/ci.yml) execute les tests, le lint (pyflakes) et le build du frontend a chaque push / pull request.

11. Livrables academiques

```
make memoir      # docs/memoire_deforestwatch.docx (page de garde, 4 chapitres,
                 # bibliographie, annexes)
make slides      # docs/soutenance_deforestwatch.pptx (16 slides)
make report      # data/processed/rapport_synthese.md (synthese chiffrée)
make guide       # GUIDE.md + docs/GUIDE.pdf (ce document)
```

12. Deploiement

```
docker-compose up --build          # API + dashboard + PostgreSQL
#   API          -> http://localhost:8000
#   Dashboard    -> http://localhost:8501
```

Production : renseignez les cles dans .env (GEE, Supabase, OpenWeather, JWT), mettez DEMO_MODE=false.
Pistes d'hébergement gratuit : dashboard sur Streamlit Community Cloud, API sur Render, base sur Supabase.

13. Depannage (FAQ)

- TensorFlow/XGBoost non installes : les modeles ont un repli automatique, tout fonctionne (le U-Net utilise des centroides spectraux, XGBoost un GradientBoosting). Installez-les pour les vraies performances.
- PostgreSQL indisponible : l'API bascule automatiquement sur SQLite local.
- Le mode 'reel' affiche la demo : aucune image dans data/raw/composites/ (lancez make export-demo ou l'export GEE).
- Google Earth Engine refuse l'accès : relancez earthengine authenticate ou verifiez le compte de service dans .env.

14. Reference rapide des commandes

Commande	Effet
make seed	Donnees de demo + entraînement
make api	Demarre l'API FastAPI (8000)
make dashboard	Demarre le dashboard Streamlit (8501)
make frontend	Demarre le frontend React (5173)
make train	Entraîne les 3 modeles + risque
make test	Lance la suite de tests
make export-demo	GeoTIFF de test dans data/raw/
python -m scripts.gee_export --drive	Export reel Sentinel-2 vers Drive
make check-data	Verifie les vraies donnees
make demo / make real	Bascule .env demo/reel
make mode	Affiche le mode courant
make memoir / make slides	Genere memoire / soutenance
make docker	Build + run via docker-compose

Genere automatiquement par scripts/generate_guide.py ? DeforestWatch-DRC, 2026.